

Supplementary Information

AI-Assisted Conversion of R Statistical Packages to Python: A Methodological Framework

Yufei Cai

Contents

Skill and Agent Architecture in Claude Code	3
Sub-Agent Specifications	3
Note analyze-r-file-dependencies	4
Note extract-r-file-language-dependencies	4
Note generate-language-dependency-conversion-guide	4
Note convert-r-function-to-python	5
Note convert-r-test-to-python	5
Note generate-python-function-tests	6
Note fix-bug-found-in-test	6
Note analyze-c-file-dependencies (supplementary, not yet invoked) . . .	6
Skill Specifications	7
Note analyze-r-folder-dependencies	7
Note extract-r-folder-language-dependencies	7
Note generate-language-dependency-conversion-guides	7
Note convert-r-file-to-python	8
Note combine-python-functions-into-file	8
Note convert-r-tests-to-python	8
Note generate-python-file-tests	8
Note fix-bugs-found-in-tests	9
Note summarize-research-report	9
Language Dependency Translation Nuances	9
Note Dependencies with Non-Trivial Numerical Semantics	9
Note Dependencies with Control-Flow or API Semantic Differences	10
Note Dependencies with Direct Python Equivalents	11
Build System and Distribution Configuration	11
Note Fortran Source Inventory	11
Note BLAS Discovery Chain	11
Note C/C++/CD Platform Coverage	12
Binary Compatibility Issues and Resolution	12
Note Bug 1: f2py Subroutine Return Value (Phase 4)	12
Note Bug 2: f2py Absorbed Dimension Arguments (Phase 6)	13
Note Bug 3: blkeyst Scalar Output Intent (Phase 7)	13
Per-Function Test Coverage Details	13
Note Positive Test Coverage	13

Note ~~N2~~ Negative Test Coverage 14

Note ~~E3~~ Edge Test Coverage 14

Note ~~D4~~ Documented Behavioural Divergences 14

Note 1 Skill and Agent Architecture in Claude Code

The methodology is implemented within the Claude Code AI development environment (Anthropic). Claude Code exposes two complementary extension mechanisms: *skills* (slash commands) and *sub-agents*. A **skill** is a top-level slash command whose behavioural specification is stored as a structured Markdown file under `.claude/commands/`. The specification uses a YAML front matter block to declare a name and one-line description, followed by a natural-language body that describes, step by step, what the skill should do: which files to read, how to partition work across sub-agents, what dispatch mode to use, and how to persist the outputs. Skills are the entry points invoked interactively by the developer; they act as orchestrators that decompose a complex, multi-step task into per-item contexts and route those contexts to the appropriate sub-agents. A **sub-agent** is a specialised worker whose specification is stored as a Markdown file under `.claude/agents/`. Like skill specifications, agent specifications use YAML front matter for name and description. The body specifies, in structured natural language, the exact inputs the agent receives, the processing steps it should perform, and the output schema it must produce. Sub-agents are dispatched by skills and do not interact with other agents directly; they receive a self-contained context bundle (file contents, JSON artefacts, conversion guides) and write their output to a specified location. The three dispatch modes used in this methodology are:

Sequential dispatch Each sub-agent invocation completes before the next begins.

Used when later invocations depend on the outputs of earlier ones (Phases 1, 4, 6, and 7a).

Parallel dispatch All sub-agent instances for a batch are launched simultaneously in a single batched call. Used when invocations are context-independent and reducing wall-clock time is the primary concern (Phase 3b: 46 conversion-guide agents launched simultaneously).

Iterative dispatch A sub-agent is repeatedly invoked against a stopping criterion.

Used in Phase 7b: the `/fix-bugs-found-in-tests` skill runs the test suite, dispatches `@fix-bug-found-in-test` on the first failing test, reinstalls the package, and repeats until zero failures remain.

The `/summarize-research-report` skill was invoked at the end of each phase to produce a structured phase summary saved to `docs/daily_summaries/`. These summaries served as the primary source material for the technical report on which the main paper is based.

Note 2 Sub-Agent Specifications

Each sub-agent is defined by a Markdown file in `.claude/agents/`. The specifications below describe each agent's inputs, processing logic, and output format as encoded in those files.

Note 2.1 analyze-r-file-dependencies

Phase invoked: Phase 1.

Inputs: A single .R source file.

Task: Parse every user-defined function in the file. For each function, classify every call in its body into one of three mutually exclusive categories: (i) *language dependencies* (calls to R built-in or package-namespaced functions, identified by not being locally defined or by carrying a namespace prefix such as `stats::`); (ii) *internal dependencies* (calls to other functions defined within the same or companion R files); (iii) *external dependencies* (calls via `.Fortran()`, `.C()`, `.Call()`, or `.External()` to compiled native-code entry points, with the entry-point symbol name recorded). Each dependency list contains unique names only.

Output: A JSON object keyed by function name. Each value is an object with three keys: "language_dependencies", "internal_dependencies", and "external_dependencies", each a JSON array of strings. The file is written to `structural_analysis/{R_subfolder}/{filename}`.

Key design decisions: The agent is instructed to treat `.Fortran()` symbol names (which may carry an `F_` prefix from the `useDynLib(.fixes)` convention) as external dependencies and to strip the prefix when recording the Fortran subroutine name, enabling direct cross-reference with the Fortran source files.

Note 2.2 extract-r-file-language-dependencies

Phase invoked: Phase 3 (first stage).

Inputs: A single .R source file and the pre-computed structural analysis JSON for that file (produced by `@analyze-r-file-dependencies`).

Task: Read the structural JSON to identify, for each function, its language dependencies (the set of R built-in or standard-library function names called). Then rescan the R source line by line, recording for each occurrence of each language dependency: (a) the dependency name, (b) the enclosing function name, (c) the line number, and (d) the complete call expression (spanning multiple lines if the call is multi-line, using a continuation-detection heuristic based on unmatched parentheses).

Output: A 4-column CSV file with columns `language_dependency`, `function_name`, `line_number`, `call_body`, written to `language_dependency_analysis/{R_subfolder}/{filename}`. One row per call-site occurrence (not per unique dependency). The agent is also instructed to write a utility script `combine_csvs.py` that merges per-file CSVs into a combined table sorted by `[language_dependency, file_path, function_name, line_number]`.

Note 2.3 generate-language-dependency-conversion-guide

Phase invoked: Phase 3 (second stage, parallel batch of 46).

Inputs: The name of a single R language dependency (e.g. `fft`), the CSV subset for that dependency (all call sites across all functions), and the R source file path for contextual reading.

Task: (1) Read the call-site lines in the R source for contextual understanding of argument types and surrounding logic. (2) Consult the R online documentation (<https://www.rdocumentation.org>) for the dependency function if its behaviour is complex or non-obvious. (3) Produce a structured Markdown guide specifying, for

each usage pattern observed in the CSV, the correct Python translation strategy with concrete before-and-after code examples. The guide must address all variants observed in the call-site data and highlight any cases where a naïve translation (e.g. using the apparent NumPy analogue) would produce silent numerical errors.

Output: A Markdown file saved to `language_dependency_analysis/conversion_guides/{dependency_name}.md`. The guide has a YAML-like header specifying the dependency name and R signature, followed by sections for each observed usage pattern.

Note 2.4 `convert-r-function-to-python`

Phase invoked: Phase 4 (sequential, 16 invocations).

Inputs: The R source fragment for a single function, the function's JSON dependency map (from the Phase 1 structural analysis), the folder of 46 language-dependency conversion guides, the output folder for the resulting JSON artefact, and (if the function has internal dependencies) the JSON artefacts of already-converted callees.

Task: Translate the R function to Python. The agent must: (1) apply the relevant conversion guides for every language dependency in the function; (2) map internal R callee calls to the Python functions documented in the callees' existing JSON artefacts; (3) translate Fortran FFI calls (`.Fortran(F_xxx, ...)`) to `f2py` invocations using the in-place output-argument convention (pre-allocating output arrays and reading them after the call); (4) produce complete type annotations using `np.ndarray[Any, np.dtype[np.float64]]` for array types; and (5) write a NumPy/SciPy-style docstring documenting parameters and return values.

Output: A JSON artefact with keys `"function_name"`, `"imports"` (a list of import statement strings), `"python_code"` (the complete function definition as a string), and `"notes"` (a list of translation decisions or caveats). Written to `conversion_results/{R_subfolder}/{function_name}.json`.

Note 2.5 `convert-r-test-to-python`

Phase invoked: Phase 6 (sequential, one invocation per R test file).

Inputs: An R test script, the R package directory (for reading the R source being tested), the Python package directory (for reading the Python implementations), and the output directory.

Task: Translate the R test script to a Python `pytest` file that: (1) replicates all function calls from the R script with the same inputs; (2) uses `rpy2` to call the R reference implementation with the same inputs and asserts numerical agreement using `numpy.testing.assert_allclose` at explicit tolerances; (3) follows the `pytest` convention of `def test_name()` functions in a file with a `test_` prefix; and (4) uses `ro.FloatVector`/`ro.IntVector` for inputs and `.rx2()` for result extraction. For tests involving external datasets (e.g. the Prestige dataset from the `carData` R package), the agent must source the data directly from R via `rpy2` to guarantee exact row correspondence.

Output: A `pytest`-compatible Python file written to the specified output directory.

Note 2.6 generate-python-function-tests

Phase invoked: Phase 7a (sequential, one invocation per public function).

Inputs: The Python function definition (signature, type annotations, docstring), the R documentation page for the corresponding R function (fetched at agent run time from the KernSmooth CRAN reference manual), and the existing test files in the test folder (to avoid duplicating coverage already present).

Task: Generate three `pytest` files: (1) *Positive tests* covering all documented parameter combinations, including cross-products of discrete parameters where relevant, integer-to-float input coercion, partial-name abbreviation (where applicable), and density/regression-output integrity checks (non-negativity, approximate integral to 1.0 for density estimators). (2) *Negative tests* covering all explicit error conditions identified in the function body and callee bodies, with assertions on both the exception type and, where the error message format is unambiguous, the message text. (3) *Edge tests* covering boundary inputs: empty arrays (where not excluded by the function’s documented constraints), single-element arrays, NaN and Inf in input, degenerate grid ranges, very small and very large bandwidths, and scale- and location-invariance properties documented in the R manual. For cases where R and Python raise different exceptions, or where one raises while the other returns NaN/Inf, the agent applies a principled loose-pass strategy (issuing `UserWarning` with a description of the divergence rather than failing the test). All tests use `rpy2` for live reference values.

Output: Three `pytest` files: `test_fn_positive.py`, `test_fn_negative.py`, `test_fn_edge.py`, written to `r2py_kernsmooth/tests/`.

Note 2.7 fix-bug-found-in-test

Phase invoked: Phase 7b (iterative, one invocation per bug).

Inputs: The full `pytest` traceback of the failing test, the relevant Python source files (`__init__.py` and, if applicable, `meson.build` and Fortran source files), and the corresponding R function implementation from `KernSmooth/R/all.R`.

Task: Diagnose the root cause of the test failure by comparing the Python implementation against the R reference. Determine whether the failure is a bug in the Python code (requiring a fix) or an inherent language-level behavioural divergence (requiring a test annotation with `UserWarning`). If a fix is required, implement it in the minimum set of files necessary and verify that the fix does not break any currently passing tests. If the fix modifies build-system files (`meson.build`, `_KernSmooth.pyf`), instruct the invoking skill to reinstall the package before retesting.

Output: Modified source files and/or test files. The skill reads the agent’s output and applies any indicated reinstall step before running the next test cycle.

Note 2.8 analyze-c-file-dependencies (supplementary, not yet invoked)

This agent is defined in the repository but was not invoked in the present project. It is the C/Fortran analogue of `@analyze-r-file-dependencies`: given a C or Fortran source file, it extracts all function definitions and classifies their calls into internal calls (other functions in the same file or project) and external calls (system library symbols,

BLAS/LAPACK symbols, or calls back into R via the R API). For the `KernSmooth` project, the Fortran sources are treated as a black box (compiled and called via `f2py` without structural analysis), so this agent was not needed. For packages whose Fortran or C code has a more complex internal structure or calls back into R, invoking this agent on the `src/` directory would produce a cross-language dependency map complementary to the R-level map from Phase 1.

Note 3 Skill Specifications

Each skill is defined by a Markdown file in `.claude/commands/`. The specifications below describe each skill's invocation syntax, processing logic, and sub-agent orchestration strategy.

Note 3.1 `/analyze-r-folder-dependencies`

Phase invoked: Phase 1.

Invocation: `/analyze-r-folder-dependencies R_folder`

Processing: Recursively scans `R_folder` for files with `.R` or `.r` extensions. For each file found, dispatches `@analyze-r-file-dependencies` sequentially. Sequential dispatch is used because the agent for each file must correctly resolve internal dependencies, and knowing the set of all function names defined across all R files (required to distinguish internal from language dependencies) is easiest when files are processed one at a time with a growing registry of known function names. The output JSON files are written to `structural_analysis/{R_folder_name}/`.

Post-processing: After all files are processed, the skill synthesises the JSON outputs with the package's `DESCRIPTION`, `NAMESPACE`, and `.Rd` documentation files into a comprehensive architecture document saved to `docs/architecture-analysis.md`.

Note 3.2 `/extract-r-folder-language-dependencies`

Phase invoked: Phase 3 (first stage).

Invocation: `/extract-r-folder-language-dependencies R_folder json_folder`

Processing: Pairs each `.R` file in `R_folder` with its corresponding JSON in `json_folder` (matching by base filename). Dispatches `@extract-r-file-language-dependencies` sequentially, one per pair. After all agents complete, invokes the `combine_csvs.py` utility script (written by the first agent invocation) to merge per-file CSVs into the combined table at `language_dependency_analysis/combined_table.csv`.

Note 3.3 `/generate-language-dependency-conversion-guides`

Phase invoked: Phase 3 (second stage).

Invocation: `/generate-language-dependency-conversion-guides R_folder combined_csv`

Processing: Reads the combined CSV and extracts all unique values of the `language_dependency` column (46 for `KernSmooth`). Launches exactly one `@generate-language-dependency-conversion-guides` instance per unique dependency in a *single parallel batch*, passing each agent its dependency name, the filtered CSV rows for that dependency, and the R source path. Parallel dispatch is used because the guides are logically independent; no guide depends

on the content of any other guide. The skill waits for all 46 agents to complete before proceeding.

Note 3.4 `/convert-r-file-to-python`

Phase invoked: Phase 4 (first stage).

Invocation: `/convert-r-file-to-python R_file json_file levels_csv guides_folder output_folder`

Processing: Parses the dependency-level CSV (*levels_csv*) to extract all functions belonging to *R_file* and derive their topological conversion order (deepest leaves first, roots last). Dispatches `@convert-r-function-to-python` *sequentially* once per function in that order. Each agent invocation writes its JSON artefact to *output_folder* before the next invocation begins. This guarantees that when the agent for function *f* is invoked, the JSON artefacts of all internal callees of *f* already exist and can be read to extract their Python signatures and return types.

Note 3.5 `/combine-python-functions-into-file`

Phase invoked: Phase 4 (second stage).

Invocation: `/combine-python-functions-into-file json_folder target_py_file`

Processing: Reads all JSON artefacts in *json_folder* in leaf-to-root order (same as conversion order). Extracts the "imports" and "python_code" fields from each artefact. Corrects three systematic error categories: (1) absolute import paths to relative imports; (2) redundant cross-function imports (co-located functions need not import each other); (3) duplicate third-party import statements (merged into canonical deduplicated forms). Assembles the final import block sorted in PEP 8 order. Writes the combined file to *target_py_file* and confirms syntax validity via `ast.parse()`.

Public API: Populates the `__all__` list from the union of public function names across all JSON artefacts (excluding underscore-prefixed functions).

Note 3.6 `/convert-r-tests-to-python`

Phase invoked: Phase 6.

Invocation: `/convert-r-tests-to-python R_test_folder R_pkg_folder py_pkg_folder output_folder`

Processing: Scans *R_test_folder* for .R scripts. Dispatches `@convert-r-test-to-python` sequentially once per script. Sequential dispatch is used to allow each agent to read the Python package structure established by prior processing. Renames output files to carry the `test_` prefix required by `pytest` for automatic test discovery if the agent does not include it.

Note 3.7 `/generate-python-file-tests`

Phase invoked: Phase 7a.

Invocation: `/generate-python-file-tests python_file`

Processing: Parses *python_file* to enumerate all function definitions and identifies public functions by cross-referencing `__all__` and the KernSmooth CRAN reference

manual. Excludes internal utility functions (those absent from the R documentation). Dispatches `@generate-python-function-tests` sequentially once per public function. Sequential dispatch allows each agent to check existing test files and avoid duplicating coverage.

Note 3.8 `/fix-bugs-found-in-tests`

Phase invoked: Phase 7b.

Invocation: `/fix-bugs-found-in-tests test_folder py_pkg_folder R_pkg_folder`

Processing: Runs `python -m pytest test_folder -q`. If all tests pass, the skill exits. Otherwise, it isolates the first failing test, extracts the full traceback, assembles a context bundle (traceback, relevant Python source, relevant R source, Fortran source if applicable), and dispatches `@fix-bug-found-in-test` with that bundle. After the agent returns, if any modified file is part of the installed package (`__init__.py`, `meson.build`, `.pyf` files), the skill runs `pip install . --no-build-isolation --force-reinstall` before running the next test cycle. The loop repeats until zero failures remain or the agent reports that the failure is an inherent language-level divergence rather than a bug.

Note 3.9 `/summarize-research-report`

Phases invoked: End of each phase (utility).

Invocation: `/summarize-research-report`

Processing: Reads the conversation context accumulated during the current session and synthesises it into a structured Markdown summary of the phase’s activities, key decisions, artefacts produced, bugs discovered and resolved, and open items. The summary is saved to `docs/daily_summaries/` with a date-stamped filename. The summaries served as the primary source material for the technical report.

Note 4 Language Dependency Translation Nuances

Phase 3 produced 46 conversion guides covering all language dependencies in `KernSmooth/R/all.R`. The eight most consequential guides (those where an incorrect translation would produce a silent numerical error) are summarised in the Methods of the main paper. This note catalogues the complete set of dependencies, grouped by translation complexity.

Note 4.1 Dependencies with Non-Trivial Numerical Semantics

These dependencies require careful translation to avoid silent numerical errors.

fft R’s `fft(x, inverse=TRUE)` is unnormalised (returns $\sum X_k e^{2\pi i k n / N}$ without dividing by N); NumPy’s `np.fft.ifft` normalises by $1/N$. The R source explicitly divides by P (or $P_1 P_2$ for 2D); these divisions must be omitted.

var R’s `var(x)` uses Bessel’s correction ($n-1$); NumPy’s `np.var(x)` does not (`ddof=0`). Use `np.std(x, ddof=1)` for all `sqrt(var(x))` occurrences.

as.double R’s pre-allocation idiom `as.double(rep(0, n))` maps to `np.zeros(n, dtype=np.float64)`.

as.integer R's `as.integer(x)` on a vector maps to `np.asarray(x).astype(np.int32)`; the 32-bit constraint is critical for Fortran FFI correctness.

sqrt Fractional exponentiation of possibly negative quantities (odd roots) requires `math.copysign(abs(x)**(1/n), x)` rather than Python's `x**(1/n)`.

matrix R's `matrix(data, nrow, ncol)` fills column-by-column (Fortran order); use `np.reshape(data, (nrow, ncol), order='F')`.

Re R's `Re(x)` on a complex vector maps directly to `np.real(x)`, but must be applied to the unnormalised inverse DFT result before or after removing the Python normalisation factor.

dnorm, dbeta Both map directly to `scipy.stats.norm.pdf` and `scipy.stats.beta.pdf` respectively, with identical parameter conventions. No numerical discrepancy.

Note 4.2 Dependencies with Control-Flow or API Semantic Differences

missing R's `missing(param)` has no direct Python equivalent. Use a `None` sentinel default with an `if param is None: guard`.

match.arg R's `match.arg(arg, choices)` supports unambiguous prefix matching. Implement as a private helper `_resolve_choice(val, choices)` performing exact match, then unique prefix match, then raising a `ValueError` with the R-equivalent message format `"'arg' should be one of ..."`.

switch R's string-key `switch(key, case1=expr1, ...)` maps to a Python dict lookup. For scalar computations, a plain dict of values is sufficient.

stop R's `stop(msg)` maps to `raise ValueError(msg)`. The error message must be reproduced verbatim (without R's function-signature prefix, which R prepends automatically).

warning R's `warning(msg)` maps to `warnings.warn(msg, UserWarning)`.

which.min, which.max R's 1-indexed result maps to `np.argmin(x) / np.argmax(x)` (0-indexed); the index is used only for array subscripting, so no offset correction is needed in practice.

quantile R's `quantile(x, 0.25)` and `quantile(x, 0.75)` with the default Type 7 interpolation method map to `np.quantile(x, 0.25, method='linear')`, which is NumPy's default. No discrepancy.

length R's `length(x)` on a vector maps to `len(x)` for 1D NumPy arrays. For 0-d arrays (produced by `np.asarray` on a scalar), use `x.ndim == 0` or `len(x) == 1` as the scalar-detection guard.

range R's `range(x)` returns a length-2 vector `c(min(x), max(x))`; maps to the Python tuple `(x.min(), x.max())` or the keyword argument `range_x=(x.min(), x.max())`.

seq R's `seq(a, b, length=n)` maps to `np.linspace(a, b, n)`.

library.dynam.unload R's package unload hook has no Python equivalent; CPython does not safely support unloading native extensions. The `.onUnload` body should be omitted entirely.

packageStartupMessage Maps to `print(..., file=sys.stderr)`, though both the `.onAttach` and `.onUnload` functions were ultimately removed as dead code in Phase 5.

Note 4.3 Dependencies with Direct Python Equivalents

The following 27 dependencies map directly to NumPy, SciPy, or Python built-ins with no semantic discrepancy: `abs` → `abs()` or `np.abs()`; `c` (vector concatenation) → `np.array([...])` or `np.concatenate`; `cumsum` → `np.cumsum`; `diff` → `np.diff`; `exp` → `np.exp`; `floor` → `np.floor`; `gamma` → `math.gamma`; `is.na` → `np.isnan`; `log` → `np.log`; `max` / `min` → `np.max` / `np.min`; `numeric` (zero-vector) → `np.zeros`; `pi` → `math.pi`; `prod` → `np.prod`; `rep` → `np.repeat` or `np.tile`; `round` → `np.round`; `sign` → `np.sign`; `sort` → `np.sort`; `sum` → `np.sum`; `trunc` → `np.trunc`; `any` → `np.any`; `all` → `np.all`; and standard R arithmetic operators mapping to NumPy element-wise equivalents.

Note 5 Build System and Distribution Configuration

Note 5.1 Fortran Source Inventory

The thirteen Fortran 77 fixed-form files compiled into the `_KernSmooth` extension are:

File	Origin
<code>src/blkest.f</code>	Original KernSmooth source
<code>src/cp.f</code>	Original KernSmooth source
<code>src/dgedi.f</code>	Original KernSmooth source (LINPACK LU)
<code>src/dgefa.f</code>	Original KernSmooth source (LINPACK LU)
<code>src/dgesl.f</code>	Original KernSmooth source (LINPACK LU)
<code>src/linbin.f</code>	Original KernSmooth source
<code>src/linbin2D.f</code>	Original KernSmooth source
<code>src/locpoly.f</code>	Original KernSmooth source
<code>src/rlbin.f</code>	Original KernSmooth source
<code>src/sdiag.f</code>	Original KernSmooth source
<code>src/sstdiag.f</code>	Original KernSmooth source
<code>src/dqrdc.f</code>	Added from Netlib (LINPACK QR, G. W. Stewart, 1978)
<code>src/dqrs1.f</code>	Added from Netlib (LINPACK QR, G. W. Stewart, 1978)

`dqrdc.f` and `dqrs1.f` are required because `blkest.f` and `cp.f` call `dqrdc_` and `dqrs1_`, respectively. The original R installation satisfies these symbols from its own internal LINPACK copy; OpenBLAS does not provide LINPACK QR routines.

Note 5.2 BLAS Discovery Chain

The BLAS discovery logic in `meson.build` follows a five-stage fallback chain designed to be portable across all target platforms while remaining functional on the HPC development cluster (Red Hat Enterprise Linux 9), where the unversioned symlink `libopenblas.so` normally installed by `openblas-openmp-devel` is absent:

1. `dependency('blas', required: false)` — queries `pkg-config`.

2. `fc.find_library('openblaso', required: false)` — OpenBLAS OpenMP variant by name.
3. `fc.find_library('openblas', required: false)` — standard OpenBLAS variant.
4. `fc.find_library('blas', required: false)` — generic BLAS name.
5. `fs.is_file('/usr/lib64/libopenblaso.so.0')` hardcoded path with `declare_dependency(['usr/lib64/libopenblaso.so.0'])`.

If all five stages fail, the build is terminated with a clear error message specifying the remediation steps for each platform.

Note 5.3 CI/CD Platform Coverage

The `python-publish.yml` GitHub Actions workflow builds wheels for up to 35 platform/Python-version combinations per release, using `cibuildwheel` on five runners:

- `ubuntu-latest` (Linux x86_64): manylinux and musllinux containers.
- `ubuntu-24.04-arm` (Linux aarch64): manylinux and musllinux containers, using a native ARM64 runner without QEMU.
- `macos-13` (macOS x86_64, Intel).
- `macos-latest` (macOS arm64, Apple Silicon).
- `windows-latest` (Windows x86_64, with MSYS2 MinGW64 toolchain).

Wheels are built for CPython 3.10 through 3.14. Windows arm64, Linux ppc64le/s390x, and free-threaded Python builds (cp313t, cp314t) are excluded; their rationale is discussed in the main paper’s Methods. OIDC trusted publishing is used for PyPI authentication, eliminating the need for stored long-lived credentials.

Note 6 f2py Compatibility Issues and Resolution

Three distinct incompatibilities between the Phase 4 conversion output and the f2py interface generated by NumPy 2.4.3 were discovered and resolved across Phases 4–7. They are catalogued here in discovery order.

Note 6.1 Bug 1: f2py Subroutine Return Value (*Phase 4*)

R’s `.Fortran()` returns a named list of all arguments; the R idiom `.Fortran(F_linbin, ...)$gcnts` extracts the modified output array from that list. The conversion agent for `linbin` generated `_KernSmooth.linbin(...)[6]`, subscripting what it modelled as a list return. An f2py-wrapped Fortran subroutine always returns `None`; output arrays are modified in-place via C pointer. The fix removed all subscript operations on Fortran subroutine return values and replaced them with reading the pre-allocated output arrays after the call. Seven functions were patched: `linbin`, `linbin2D`, `blkest`, `cpblock`, `locpoly`, `sdiag`, and `sstdiag`. Scalar output arguments additionally required replacement of `np.float64(0.0)` placeholders with `np.zeros(1, dtype=np.float64)`.

Note 6.2 Bug 2: f2py Absorbed Dimension Arguments (*Phase 6*)

NumPy 2.4.3's f2py absorbs Fortran dimension scalar parameters (integers whose sole role is to specify the length of an array argument) out of the required positional argument list, converting them into optional parameters with defaults inferred from array shapes. The Phase 4 code still injected these integers at their original Fortran-order positions, causing a systematic argument misalignment: every argument after each absorbed dimension was shifted by one position, producing shape-check failures or silently incorrect results. Additionally, 2D Fortran arrays must be passed as 2D NumPy arrays with `order='F'`; the prior code flattened them to 1D, which was valid in earlier f2py versions. Five subroutine call sites were corrected: `locpol` (3 absorbed arguments, 2D arrays added), `sdiag` (3 absorbed, 2D arrays), `sstdg` (3 absorbed, 2D arrays), `blkest` (2 absorbed), and `cp` (3 absorbed).

Note 6.3 Bug 3: blkest Scalar Output Intent (*Phase 7*)

The three scalar output arguments of `blkest` (`sigsqe`, `th22e`, `th24e`) were declared by f2py as `input float` (passed by value) because they appear as plain Fortran `DOUBLE PRECISION` scalars without array syntax. The Fortran subroutine writes to them by direct assignment, but f2py passed scalar copies, so the writes never propagated back to Python. Consequently, `blkest` always returned zeros for all three outputs, causing `dpill` to divide by zero in an intermediate bandwidth computation, propagating NaN through the remainder of `dpill`. The fix comprised three coordinated changes: (1) generating a full f2py signature file `r2py_kernsmooth/src/_KernSmooth.pyf` with explicit `intent(out)` annotations on the three scalars; (2) updating `meson.build` to pass `_KernSmooth.pyf` as the sole f2py input; and (3) updating `blkest()` in `__init__.py` to unpack the three scalars as a return tuple from `_KernSmooth.blkest()`.

Note 7 Per-Function Test Coverage Details

Note 7.1 Positive Test Coverage

Each public function's positive tests form a combinatorial grid over its discrete parameters. Highlights:

- bkde** All 5 kernel types (`normal`, `box`, `epanechnikov`, `biweight`, `triweight`), both canonical modes, auto-computed and explicit bandwidth, varied `gridsize` and `range_x`, `truncate` mode, integer input coercion, and partial kernel-name abbreviations (e.g. "n" matching "normal").
- bkde2D** Bivariate normal input, various `gridsize` combinations, asymmetric and scalar bandwidths, explicit `range_x`, `truncate` mode, integer coercion, and density integration.
- bkfe** All even derivative orders `drv` $\in \{0, 2, 4, 6, 8, 10\}$, varied sample sizes and distributions, and `binned` mode.
- dpih** All three `scalest` options $\times$ all six `level` values (18 combinations) and an analytical formula check for `level=0`.
- dpik** All 5 kernels $\times$ 6 levels (30 combinations) and all 3 `scalest` $\times$ 6 levels (18

combinations), plus `canonical` mode and partial-name abbreviations. Includes a kernel bandwidth ordering invariant for the non-canonical case.

dpill Standard regression scenarios, various `blockmax`, `divisor`, `trim`, `proptrun`, and `gridsize` settings, and multiple regression function types.

locpoly Regression with `drv` $\in \{0, 1, 2, 3\}$, scalar and vector bandwidth, `truncate` and `binned` modes, density mode (`y=None`), and the Prestige dataset canonical example from the KernSmooth reference manual.

Note 7.2 Negative Test Coverage

Negative tests cover all explicit error conditions processed in the function body and related helpers. Where both R and Python raise but with different message phrasing (R prepends the function-call context, Python does not), the test issues a `UserWarning` documenting the divergence. Where R raises inside a callee (`seq.default()`, `seq.int()`) at a lower stack level and Python propagates NaN via IEEE 754 arithmetic instead, the test verifies the Python NaN-return behaviour and issues a `UserWarning`.

Note 7.3 Edge Test Coverage

Edge tests cover: $n = 1$ (single observation) for all functions; $n = 2$ (minimum for meaningful density estimation); constant data (all observations identical); NaN and Inf values in input; degenerate `range_x` ($a = b$); very small bandwidth ($h = 10^{-8}$); very large bandwidth ($h = 10^8$); power-of-two `gridsize` regression guard (carried forward from Phase 6 for `bkde` and `bkfe`); scale-invariance and location-shift invariance (tested for `dpih` and `dpik` against live R reference values); and the NaN-equals-NaN edge case for `dpill` (both Python and R legitimately return NaN for ill-conditioned seeds, and the assertion uses `np.isnan(h1)` and `np.isnan(h2)` rather than `pytest.approx`).

Note 7.4 Documented Behavioural Divergences

Five cases of inherent Python-vs-R divergence are documented in the test suite as `UserWarning` rather than treated as failures:

1. `bkde` with degenerate `range_x` ($a = b$): R returns an array of NaN; Python raises `ZeroDivisionError`.
2. `bkfe` with Inf in input: R raises inside `seq.default()`; Python propagates NaN via IEEE 754.
3. `dpih` with `level=-1`: R returns `numeric(0)` (a zero-length vector, a valid R type); Python raises `UnboundLocalError` because no branch assigns the result variable for a negative level.
4. `dpill` with `divisor=0`: R returns a valid result (integer division yields Inf at an intermediate step, which does not propagate as an error); Python raises `ZeroDivisionError`.
5. `bkde2D` with Inf in input: R raises inside `seq.default()`; Python returns a partially non-finite result.

Supplementary Figures

[Supplementary Figure S1 placeholder]

*KernSmooth function dependency graph. A directed acyclic graph with three horizontal layers. Level 2 (bottom): two nodes, **linbin** and **rlbin**, each shown with a dashed arrow leading to a Fortran subroutine icon (**F_linbin** and **F_rlbin** respectively). Level 1 (middle): seven nodes (**bkfe**, **blkest**, **cpblock**, **linbin2D**, **locpoly**, **sdiag**, **sstdiag**), each with dashed arrows to their respective Fortran subroutines and solid arrows up from Level 2 callees. Level 0 (top): seven nodes (**.onAttach**, **.onUnload**, **bkde**, **bkde2D**, **dpih**, **dpik**, **dpill**). Solid arrows from Level 0 to Level 1 nodes indicate internal calls. Exported functions (all Level 0 nodes and **bkfe**) are shown with a double border. The **linbin** node has seven incoming arrows from callers, highlighted with a thicker line to indicate its centrality. A legend distinguishes exported (double border), internal (single border), and Fortran entry point (rounded box) node styles.*

Figure S1: **Supplementary Fig. S1: KernSmooth three-level function dependency graph.** Level 0 nodes are user-callable entry points; Level 1 nodes are intermediate helpers; Level 2 nodes are the fundamental binning primitives. Dashed arrows indicate calls to compiled Fortran subroutines. **linbin** is the most central function, serving as the binning primitive for seven distinct callers spanning all three dependency levels.

[Supplementary Figure S2 placeholder]

f2py calling convention comparison. Two parallel columns, labelled “R (.Fortran interface)” and “Python (f2py interface)”. R column: pseudocode showing (1) `.Fortran("linbin", x=x, ..., gcnts=as.double(rep(0,M)), ...)`, (2) an arrow labelled “returns named list of ALL arguments”, (3) `result$gcnts` subscript to extract output. Python column: pseudocode showing (1) `gcnts = np.zeros(M, dtype=np.float64)`, (2) `_KernSmooth.linbin(x, ..., gcnts, ...)`, (3) an arrow labelled “returns None; gcnts modified in-place via C pointer”, (4) `gcnts` used directly. A second comparison row below shows the scalar output case: R sends `sigsqe=0.0` and receives `result$sigsqe`; Python must send `sigsqe=np.zeros(1)` and read `sigsqe[0]`, or alternatively use `intent(out)` in the `.pyf` file to receive `sigsqe` as a return value. A note at the bottom highlights the absorbed-dimension issue: `f2py` removes dimension integers from the positional argument list, so `n` in `blkest(n, ...)` becomes optional.

Figure S2: **Supplementary Fig. S2: Comparison of R `.Fortran()` and Python `f2py` calling conventions.** In R, `.Fortran()` returns a named list of all arguments, including output arrays that were modified in-place by the Fortran subroutine; the output is extracted by name. In Python, the `f2py`-wrapped subroutine returns `None`; output arrays must be pre-allocated and read after the call. Scalar outputs require either a length-1 array workaround or explicit `intent(out)` declarations in a `.pyf` signature file. Dimension scalar arguments are absorbed by `f2py` from the positional argument list in NumPy 2.4.3.